

边界校验类

场景 B-1：MatterContext 未组装完毕，Runtime 拒绝触发

前置条件：

Controller 投递了一条级联触发消息，但 MatterContext 缺失：flowId、matterId；
assigneeMapping 在首次触发时一定为空 Map；只记录已触发的 Human 节点的解析结果，未触发的 Human 节点不记录；
context 不做校验：可能为空 Map 也有可能不空；

触发事件：

Runtime 消费到级联触发消息

Runtime 行为：

1. 从消息体中取 matterId，为空则写入错误日志，终止执行
2. 用 matterId 从 MongoDB 读取 MatterContext，
读取失败或 flowId 缺失则写入错误日志，终止执行
3. 用 flowId 从 MongoDB 读取 FlowDefinition，
读取失败则写入错误日志，终止执行

后置状态：

MatterContext 保持收到消息时的原始状态
错误日志已写入，等待 FDE 排查

异常分支：

无，本场景本身就是异常分支的终态

协作边界：

Controller：保证投递消息前 MatterContext 已完整组装，
这是 Controller 对 Runtime 的契约
保证 flowId 指向一个真实存在的 Flow 版本
RocketMQ：消息消费后立即 ACK，不依赖重投
Runtime：只做校验和日志，不做任何修复尝试
不假设 FlowDefinition 一定存在，读取失败即终止
Asset：保证 FlowDefinition 在被引用前已存在于 MongoDB

场景 B-2：级联触发消息指向的 nodeId 在 FlowDefinition 中不存在

前置条件：

MatterContext 校验通过，FlowDefinition 读取成功，
但消息中的 nodeId 在 FlowDefinition.nodes 中找不到

触发事件：

Runtime 尝试定位当前节点

Runtime 行为：

1. 从 `FlowDefinition.nodes` 按 `nodeId` 查找节点，未找到则写入错误日志（记录 `matterId`、`nodeId`），终止执行

后置状态：

`MatterContext` 保持不变
错误日志已写入，等待 FDE 排查

异常分支：

无

协作边界：

`Controller/TriggerActions`：保证投递的 `nodeId` 来自 `FlowDefinition` 的合法节点，不允许构造不存在的 `nodeId`
`Runtime`：不假设 `nodeId` 一定合法，找不到即终止

正常执行类

场景 N-1：START 节点触发，Matter 开始执行

前置条件：

`MatterContext` 已完整组装：`matterId`、`flowId` 已持久化，`context` 已装载 `Matter` 发起人输入，`assigneeMapping` 为空 `Map`，`executionStates` 为空 `Map`

触发事件：

`Controller` 向 `RocketMQ` 投递 `CascadeExecuteMessage`（`matterId`、`nodeId`=START节点id、`triggerType`=FIRST_TRIGGER）

`Runtime` 行为：

1. 消费消息，从消息体取 `matterId`、`nodeId`、`triggerType`
2. 校验 `MatterContext` 和 `FlowDefinition` 完整性（见 B-1）
3. 定位 START 节点
4. 幂等守卫：`triggerType`=FIRST_TRIGGER，`executionStates` 中 START 节点无记录则放行，有任何记录则终止执行
5. 【副作用】持久化：原子写入 START 节点状态 TRIGGERED，从消息元数据中取 `queueId` 写入 `MatterContext`，写入当前 `cursor`，ACK 消息
6. 内存求值 `triggers`，确定满足条件的目标节点
7. 【副作用】持久化：原子写入 START 节点状态 COMPLETED，写入当前 `cursor`
8. 【副作用】持久化：写入当前 `cursor`，使用事务消息投递目标节点 `CascadeExecuteMessage`（`triggerType`=FIRST_TRIGGER），`cursor` 持久化作为本地事务，`commit` 后消息对消费方可见

后置状态：

`executionStates` 中 START 节点状态为 COMPLETED

`cursor` 已持久化至步骤8

后续节点的 `CascadeExecuteMessage` 已投递到 RocketMQ

异常分支:

triggers 条件全部不满足: **Matter** 无后续节点, 执行自然终止

步骤5前崩溃: 消息未 ACK, RocketMQ 重投,

幂等守卫检查无记录, 放行, 重新执行

步骤5后、步骤7前崩溃: **RecoveryRunner** 投递

`CascadeExecuteMessage` (`triggerType=RECOVERY`),

由正常消费流程从 `cursor` 位置恢复执行

步骤7后、步骤8前崩溃: 恢复后从 `cursor` 位置继续,

重新执行步骤8, 事务消息保证消息不丢失

步骤8后崩溃: 子节点消息已在 RocketMQ 中, 自然被消费

协作边界:

Controller: 负责投递 **START** 节点初始触发消息,

保证 `triggerType=FIRST_TRIGGER`

RocketMQ: `at-least-once` 覆盖消费到写 **TRIGGERED** 的窗口,

事务消息保证多出度消息投递原子性

Runtime: **ACK** 在步骤5之后立即执行, 步骤5后的恢复

依赖 `cursor` 机制, 不依赖 RocketMQ 重投,

禁止直接创建虚拟线程触发子节点

场景 N-2: TASK 节点首次触发

前置条件:

上游节点已 **COMPLETED**,

`CascadeExecuteMessage` (`triggerType=FIRST_TRIGGER`) 已投递,

`executionStates` 中当前 **TASK** 节点无记录

触发事件:

Runtime 消费到当前 **TASK** 节点的 `CascadeExecuteMessage`

Runtime 行为:

1. 消费消息, 从消息体取 `matterId`、`nodeId`、`triggerType`
2. 校验 **MatterContext** 和 **FlowDefinition** 完整性 (见 B-1)
3. 定位当前 **TASK** 节点
4. 幂等守卫: `triggerType=FIRST_TRIGGER`,
`executionStates` 中当前节点无记录则放行,
有任何记录则终止执行
5. 【副作用】持久化: 原子写入节点状态 **TRIGGERED**,
写入当前 `cursor`, **ACK** 消息
6. 从 MongoDB 读取 **Task** 资产
7. 按 `input_mapping` 从 `context` 取值, 构造 **RPC** 入参
8. 【副作用】执行 **RPC** 调用, `output` 暂存内存,
持久化: 按 `output_schema` 将 `output` 写入 `context`,
写入节点状态 **COMPLETED**, 写入当前 `cursor`
9. 【副作用】持久化: 写入当前 `cursor`,
使用事务消息投递目标节点 `CascadeExecuteMessage`
(`triggerType=FIRST_TRIGGER`), `cursor` 持久化作为本地事务,

`commit` 后消息对消费方可见

后置状态:

`executionStates` 中当前节点状态为 `COMPLETED`

`cursor` 已持久化至步骤9

`context` 中已写入 `Task output`

后续节点 `CascadeExecuteMessage` 已投递

异常分支:

`Task` 资产不存在: 写入错误日志, 终止执行 (系统异常)

`input_mapping` 引用的 `key` 不存在: 写入错误日志,
终止执行 (系统异常)

步骤5前崩溃: 消息未 `ACK`, `RocketMQ` 重投,
幂等守卫检查无记录, 放行, 重新执行

步骤5后、步骤8前崩溃: `RecoveryRunner` 投递
`CascadeExecuteMessage (triggerType=RECOVERY)`,
由正常消费流程按 `Task.idempotency` 决定:
`SAFE` → 从 `cursor` 位置自动恢复执行
`UNSAFE` → 写入节点状态 `ERROR`, 等待人工触发 `TASK_RETRY`

步骤8后、步骤9前崩溃: 恢复后从 `cursor` 位置继续,
重新执行步骤9, 事务消息保证消息不丢失,
下游幂等守卫兜底

`RPC` 调用失败: 见失败处理类场景

协作边界:

`Runtime`: `ACK` 在步骤5之后立即执行, 步骤5后的恢复
依赖 `cursor` 机制, 不依赖 `RocketMQ` 重投,
禁止直接创建虚拟线程触发子节点

`MongoDB`: 提供 `Task` 资产读取和 `MatterContext` 原子写回

`RocketMQ`: `at-least-once` 覆盖消费到写 `TRIGGERED` 的窗口,
事务消息保证多出度消息投递原子性

场景 N-3: HUMAN 节点触发

前置条件:

上游节点已 `COMPLETED`,

`CascadeExecuteMessage (triggerType=FIRST_TRIGGER)` 已投递,

`executionStates` 中当前 `HUMAN` 节点无记录,

`assigneeMapping` 中当前节点无记录

触发事件:

`Runtime` 消费到当前 `HUMAN` 节点的 `CascadeExecuteMessage`

`Runtime` 行为:

1. 消费消息, 从消息体取 `matterId`、`nodeId`、`triggerType`
2. 校验 `MatterContext` 和 `FlowDefinition` 完整性 (见 B-1)
3. 定位当前 `HUMAN` 节点
4. 幂等守卫: `triggerType=FIRST_TRIGGER`,
`executionStates` 中当前节点无记录则放行,
有任何记录则终止执行

5. 调用 Roster，将 HUMAN 节点的 assignee 描述符数组解析为责任人绑定结果，暂存内存
6. 【副作用】持久化：原子写入节点状态 TRIGGERED，写入 assigneeMapping[nodeId]，写入 cursor，ACK 消息
7. 【副作用】持久化：原子写入节点状态 WAITING，写入 cursor，投递 BusinessNotificationMessage 到通知通道，通知责任人介入

后置状态：

executionStates 中当前节点状态为 WAITING
assigneeMapping 中已记录当前节点的责任人绑定结果
cursor 已持久化至步骤7
BusinessNotificationMessage 已投递到通知通道

异常分支：

Roster 调用失败（描述符找不到解析规则）：
写入节点状态 ERROR，投递 SystemErrorMessage 到通知通道，
抛出异常，写入日志，消息未 ACK，等待 FDE 修复后
通过前端重新触发
步骤6前崩溃：消息未 ACK，RocketMQ 重投，
幂等守卫检查无记录，放行，重新执行
步骤6后、步骤7前崩溃：RecoveryRunner 扫描到节点状态为
TRIGGERED，从 cursor 位置恢复，重新执行步骤7，
BusinessNotificationMessage 可能重复投递，
通知消费端自行保证幂等
步骤7后崩溃：节点已为 WAITING，通知消息已在通知通道，
自然被消费端处理

协作边界：

Controller：保证 triggerType=FIRST_TRIGGER，
负责将责任人提交数据写入 MatterContext
并投递 HUMAN_RESUME 消息（见 N-4）
Roster：解析 assignee 描述符为责任人绑定结果，
按负载均衡（最少待处理事件）选出自然人，
找不到解析规则时报错
RocketMQ：at-least-once 覆盖消费到写 TRIGGERED 的窗口；
通知通道不做事务消息，通知消费端自行幂等
Runtime：ACK 在步骤6之后立即执行，
HUMAN 节点不触发子节点，等待 Controller 驱动唤醒

场景 N-4：HUMAN 节点唤醒（责任人提交数据）

前置条件：

当前 HUMAN 节点状态为 WAITING，
assigneeMapping 中已记录当前节点的责任人绑定结果，
责任人已通过前端提交结构化数据

触发事件：

Controller 将责任人提交数据写入 MatterContext.context，
向 RocketMQ 投递 CascadeExecuteMessage

(matterId、nodeId、triggerType=HUMAN_RESUME)

Runtime 行为:

1. 消费消息，从消息体取 matterId、nodeId、triggerType
2. 校验 MatterContext 和 FlowDefinition 完整性（见 B-1）
3. 定位当前 HUMAN 节点
4. 幂等守卫: triggerType=HUMAN_RESUME，
executionStates 中当前节点状态为 WAITING 则放行，
其他状态则终止执行
5. 【副作用】持久化: 原子写入节点状态 TRIGGERED，
写入 cursor，ACK 消息
6. 内存求值 triggers，确定满足条件的目标节点
7. 【副作用】持久化: 原子写入节点状态 COMPLETED，
写入 cursor，使用事务消息投递目标节点
CascadeExecuteMessage (triggerType=FIRST_TRIGGER)，
cursor 持久化作为本地事务，
commit 后消息对消费方可见

后置状态:

executionStates 中当前节点状态为 COMPLETED
cursor 已持久化至步骤7
后续节点的 CascadeExecuteMessage 已投递到 RocketMQ

异常分支:

步骤5前崩溃: 消息未 ACK，RocketMQ 重投，
幂等守卫检查状态为 WAITING，放行，重新执行
步骤5后、步骤7前崩溃: RecoveryRunner 扫描到节点状态为
TRIGGERED，从 cursor 位置恢复，重新执行步骤7，
事务消息保证消息不丢失
步骤7后崩溃: 子节点消息已在 RocketMQ 中，自然被消费
triggers 条件全部不满足: Matter 无后续节点，执行自然终止

协作边界:

Controller: 负责校验责任人身份与提交数据合法性，
保证数据写入 MatterContext 后再投递
HUMAN_RESUME 消息，两步之间非原子，
Controller 自行保证顺序
RocketMQ: at-least-once 覆盖消费到写 TRIGGERED 的窗口，
事务消息保证多度消息投递原子性
Runtime: ACK 在步骤5之后立即执行，
步骤5后的恢复依赖 cursor 机制，
不依赖 RocketMQ 重投

场景 N-5：多出度条件触发（通用子流程）

说明:

本场景描述所有节点类型在执行完毕后的出度触发逻辑，
N-1、N-2、N-4 中的触发投递步骤均引用本场景，
不单独重复描述。

前置条件:

当前节点已执行完毕，准备求值 `triggers`，
`MatterContext.context` 中已有求值所需的数据

触发事件:

`Runtime` 对当前节点的 `triggers` 数组求值

Runtime 行为:

1. 遍历 `triggers` 数组，对每个元素求值：
 `condition` 为 `null` 则无条件满足，
 否则递归求值 `ConditionNode` 树
2. 收集所有满足条件的 `target` 节点 ID，暂存内存
3. 【副作用】持久化：原子写入当前节点状态 `COMPLETED`，
 写入 `cursor`，使用事务消息批量投递所有满足条件的
 目标节点 `CascadeExecuteMessage`
 (`triggerType=FIRST_TRIGGER`)，
 `cursor` 持久化作为本地事务，
 `commit` 后消息对消费方可见

后置状态:

当前节点状态为 `COMPLETED`
所有满足条件的目标节点 `CascadeExecuteMessage` 已投递
不满足条件的出度不产生任何消息

异常分支:

所有 `condition` 均不满足：`Matter` 无后续节点，执行自然终止，
不投递任何消息，不写入任何异常状态
`condition` 求值异常（`value` 类型不匹配等）：
 写入节点状态 `ERROR`，
 投递 `BusinessNotificationMessage` 通知 `Flow` 设计者修复，
 抛出异常，写入日志
步骤3前崩溃：从 `cursor` 位置恢复，重新求值并投递，
 下游幂等守卫兜底
步骤3后崩溃：所有目标节点消息已在 `RocketMQ` 中，
 自然被消费

协作边界:

Flow 设计者：保证 `condition` 表达式合法且可求值，
 保证出度覆盖完整，避免所有条件均不满足
 导致 `Matter` 静默终止
Runtime：只负责求值和投递，不校验 `condition` 语义正确性，
 不对静默终止做任何干预

场景 N-6：并发执行（同一 Matter 多个子节点同时触发）

说明:

本场景描述 `Flow` 树中存在多出度时，多个子节点的
`CascadeExecuteMessage` 被同时投递后的并发执行情况。
由于同一 `Matter` 的所有消息归属于同一 `MessageQueue`，
多个子节点的消息由同一 `Runtime` 实例顺序消费，

但各子节点的执行通过虚拟线程并发进行，在时间上存在重叠。

前置条件：

上游节点已 `COMPLETED`，
多个子节点的 `CascadeExecuteMessage` 已批量投递，
各子节点在 `executionStates` 中均无记录

触发事件：

同一 `Runtime` 实例顺序消费多个子节点的
`CascadeExecuteMessage`，为每个子节点启动独立虚拟线程
并发执行

`Runtime` 行为：

各子节点独立执行，互不感知，每个子节点均走完整的
执行流程（见 `N-2` 或 `N-3`），包括：

1. 幂等守卫独立校验各自节点状态
2. 各自独立持久化，字段级原子写入互不干扰
3. 各自独立投递下游消息

后置状态：

各子节点独立完成执行，状态互不影响
`MatterContext.context` 中各节点的 `output` 独立写入，
`key` 格式为 `{node_id}.{field_name}`，天然不冲突

异常分支：

某个子节点执行失败：只影响该节点及其下游，
不影响其他并发执行的子节点
同一节点被重复投递：幂等守卫拦截，终止执行

协作边界：

`Runtime`：同一 `MessageQueue` 保证同一 `Matter` 由同一实例消费，
字段级原子写入（`$set`）保证并发写入安全，
不需要跨节点协调，不需要分布式锁
`MongoDB`：`$set` 原子操作天然支持并发写入不同字段，
`key` 命名规范保证不同节点的 `output` 不冲突

失败处理类

场景 F-1：TASK RPC 调用失败

前置条件：

`TASK` 节点已写入 `TRIGGERED` 状态，
`Task` 资产读取成功，`input_mapping` 构造入参成功，
`RPC` 调用已发出

触发事件：

`RPC` 调用返回失败，包括：
超时、网络异常、服务端返回错误码

Runtime 行为:

1. 捕获 RPC 异常或错误码
2. 按 Task.policy 判断是否重试:
 - UNSAFE: retry_max_attempts 强制为1, 直接跳转步骤4
 - SAFE: 当前重试次数未达 retry_max_attempts, 等待 retry_backoff_ms 后重新执行 RPC, 重试次数已达上限则跳转步骤4
3. 重试成功: 按 N-2 步骤8继续执行
4. 重试耗尽或 UNSAFE 直接失败:
 - 【副作用】持久化: 写入节点状态 ERROR,
 - 写入 message (userMessage), 写入 cursor,
 - 投递 BusinessNotificationMessage 通知业务人员介入,
 - 投递 SystemErrorMessage 通知 FDE 排查下游服务健康状况

后置状态:

executionStates 中当前节点状态为 ERROR
 message 已写入失败原因
 cursor 已持久化
 BusinessNotificationMessage 已投递到通知通道
 Matter 执行暂停, 等待人工触发 TASK_RETRY

异常分支:

熔断器已触发: 不执行 RPC, 直接进入步骤4 (见 F-3)
 重试过程中崩溃: RecoveryRunner 投递
 CascadeExecuteMessage (triggerType=RECOVERY),
 由正常消费流程按 Task.idempotency 决定恢复策略:
 SAFE → 从 cursor 位置恢复, 重试次数重置为
 retry_max_attempts 重新开始计数,
 崩溃本身不消耗重试次数
 UNSAFE → 写入节点状态 ERROR, 等待人工触发 TASK_RETRY

协作边界:

Runtime: 重试逻辑在同一虚拟线程内执行,
 不重新投递消息, 不依赖 RocketMQ 重投,
 重试次数在内存中计数, 崩溃后从 cursor 恢复
 Task 资产: policy 中声明 retry_max_attempts、
 retry_backoff_ms, Runtime 严格遵守
 业务人员: 收到通知后判断是否触发 TASK_RETRY (见 F-2)

场景 F-2: TASK 重试 (人工触发)

前置条件:

当前 TASK 节点状态为 ERROR,
 业务人员已通过前端触发重试

触发事件:

Controller 向 RocketMQ 投递 CascadeExecuteMessage
 (matterId、nodeId、triggerType=TASK_RETRY)

Runtime 行为:

1. 消费消息，从消息体取 `matterId`、`nodeId`、`triggerType`
2. 校验 `MatterContext` 和 `FlowDefinition` 完整性（见 B-1）
3. 定位当前 `TASK` 节点
4. 幂等守卫: `triggerType=TASK_RETRY`，
`executionStates` 中当前节点状态为 `ERROR` 则放行，
其他状态则终止执行
5. 【副作用】持久化: 原子写入节点状态 `TRIGGERED`，
清空 `message`，写入 `cursor`，`ACK` 消息
6. 从 `MongoDB` 读取 `Task` 资产
7. 按 `input_mapping` 从 `context` 取值，构造 `RPC` 入参
8. 【副作用】执行 `RPC` 调用，`output` 暂存内存，
持久化: 按 `output_schema` 将 `output` 写入 `context`，
写入节点状态 `COMPLETED`，写入 `cursor`
9. 【副作用】持久化: 写入 `cursor`，
使用事务消息投递目标节点 `CascadeExecuteMessage`
（`triggerType=FIRST_TRIGGER`），`cursor` 持久化作为本地事务，
`commit` 后消息对消费方可见

后置状态:

`executionStates` 中当前节点状态为 `COMPLETED`
`cursor` 已持久化至步骤9
`context` 中已写入 `Task output`
 后续节点 `CascadeExecuteMessage` 已投递

异常分支:

`RPC` 调用再次失败: 见 F-1，重新进入失败处理流程
 步骤5前崩溃: 消息未 `ACK`，`RocketMQ` 重投，
 幂等守卫检查状态为 `ERROR`，放行，重新执行
 步骤5后、步骤8前崩溃: `RecoveryRunner` 投递
`CascadeExecuteMessage`（`triggerType=RECOVERY`），
 由正常消费流程按 `Task.idempotency` 决定恢复策略:
`SAFE` → 从 `cursor` 位置恢复，重试次数重置为
`retry_max_attempts` 重新开始计数
`UNSAFE` → 写入节点状态 `ERROR`，等待人工再次触发 `TASK_RETRY`
 步骤8后、步骤9前崩溃: 恢复后从 `cursor` 位置继续，
 重新执行步骤9，事务消息保证消息不丢失
 步骤9后崩溃: 子节点消息已在 `RocketMQ` 中，自然被消费

协作边界:

Controller: 负责校验业务人员身份与触发权限，
 保证 `triggerType=TASK_RETRY`
RocketMQ: `at-least-once` 覆盖消费到写 `TRIGGERED` 的窗口，
 事务消息保证多出度消息投递原子性
Runtime: `ACK` 在步骤5之后立即执行，
 步骤5后的恢复依赖 `cursor` 机制，
 不依赖 `RocketMQ` 重投

场景 F-3: 熔断触发

前置条件:

当前 TASK 节点准备执行 RPC 调用，
熔断器已处于 OPEN 状态（连续失败次数达到 `cb_failure_threshold`）

触发事件:

Runtime 尝试执行 RPC 调用，熔断器拦截

Runtime 行为:

1. 执行 RPC 前检查熔断器状态，
熔断器为 OPEN 则直接拦截，不发出 RPC 请求
2. 【副作用】持久化：写入节点状态 ERROR，
写入 message（说明熔断器已触发），写入 cursor，
投递 `BusinessNotificationMessage` 通知业务人员介入，
投递 `SystemErrorMessage` 通知 FDE 排查下游服务健康状况

后置状态:

`executionStates` 中当前节点状态为 ERROR
message 已写入熔断原因
cursor 已持久化
`BusinessNotificationMessage` 已投递到通知通道
Matter 执行暂停，等待人工触发 TASK_RETRY

异常分支:

熔断器处于 HALF_OPEN 状态（`cb_recovery_ms` 已过）：
允许一次试探性 RPC 调用，
成功则关闭熔断器，继续正常执行流程，
失败则重新进入 OPEN 状态，写入节点状态 ERROR

协作边界:

Runtime: 熔断器按 Task 维度独立维护，
不同 Task 的熔断器互不影响，
熔断器状态在内存中维护，实例重启后重置，
`cb_failure_threshold`、`cb_recovery_ms`
从 `Task.policy` 中读取
业务人员: 收到通知后等待下游服务恢复，
再触发 TASK_RETRY（见 F-2），
TASK_RETRY 时熔断器若仍为 OPEN 则再次拦截
FDE: 若熔断频繁触发，需检查下游服务健康状况
或调整 `Task.policy` 中的熔断参数